

Closed ranges may be a problem; breaking counted_iterator is not the solution

Document #: P2799R0
Date: 2023-02-14
Project: Programming Language C++
Audience: SG9
LEWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

§ 1 Abstract

The [US 46-107/\[P2406R5\]](#) complaint boils down to a desire to adapt a closed range - something foreign to the C++ iterator model - into a half-open range, and making breaking changes to counted_iterator is the wrong way to solve that problem.

§ 2 C++ ranges are half-open

C++ ranges are half-open.

This is fundamental to the C++ iterator model. As 25.3.1 [\[iterator.requirements.general\]](#) p7 explains:

Just as a regular pointer to an array guarantees that there is a pointer value pointing past the last element of the array, so for any iterator type there is an iterator value that points past the last element of a corresponding sequence. Such a value is called a *past-the-end* value.

A range of N elements has $N + 1$ unique iterator values. N values corresponding to the N elements, and the past-the-end value. This is iterator 101.

C++20 introduced the notion of *counted ranges*, but the same principle still applies. 25.3.1 [\[iterator.requirements.general\]](#) p11:

A *counted range* $i + [0, n)$ is empty if $n == 0$; otherwise, $i + [0, n)$ refers to the n elements in the data structure starting with the element pointed to by i and up to but not including the element, if any, pointed to by the result of n applications of $++i$. A counted range $i + [0, n)$ is *valid* if and only if $n == 0$; or n is positive, i is dereferenceable, and $++i + [0, --n)$ is valid.

§ 3 What does counted_iterator do?

It attaches an integer count to an iterator:

```
template <input_or_output_iterator It>
struct counted_iterator {
    It it;
    int count;
};
```

Nothing less, nothing more. `count` changes as it moves, allowing the distance between two iterators to the same range to be easily computed.

Counting down instead of up (i.e., increment is `++it; --count;` and decrementing is `--it; ++count;`) allows the past-the-end test for a simple counted range to be expressed as `count == 0` regardless of what the number of elements in the range is, but that's just an implementation detail to allow for the convenience of a stateless sentinel. It could have incremented `count` instead and have the past-the-end check be `count == N`, at the cost of a stateful sentinel that stores `N`. `take_view`, of course, adds its own sentinel in many cases, because the count we are given is the maximum number of elements to be taken, not the exact number: its test for past-the-end is `it == end or count == 0`.

§ 4 Closed ranges

As it turns out, people occasionally find the requirement of a past-the-end value too constraining. They want to represent a range of N elements with just N iterator values - one for each element. In other words, they want a closed range, not a half-open one.

Consider the two examples given in [\[P2406R5\]](#):

- `iota(0) | filter([](auto i) { return i < 11; })`: This “range”¹ has 11 elements, and 11 iterator values. The attempt to compute the nonexistent 12th iterator value is what causes the problem.
- The `istream_view` example:

```
auto iss = std::istringstream("0 1 2");
for (auto i : rn::istream_view<int>(iss)
     | rv::take(1))
    std::cout << i << '\n';
auto i = 0;
iss >> i;
assert(i == 1); // FAILS, i == 2
```

Here, the actual problem is that `istream_view` expects all access to the stream to happen through its iterators. Nothing is actually lost if its iterator is consistently used. In the example above, we can extract the `istream_view` iterator out of the past-the-end `counted_iterator` using the allegedly useless `base()` function:

```
auto iss = std::istringstream("0 1 2");
auto tv = rn::istream_view<int>(iss)
         | rv::take(1);
auto it = tv.begin();
for (; it != tv.end(); ++it)
    std::cout << *it << '\n';
auto next = std::move(it).base();
```

```
auto i = *next;
assert(i == 1); // OK
```

But since the code wants to break `istream_view`'s expectation and rely on what is basically an implementation detail², we are left again with a range of N elements that needs to be represented using N iterator values.³

Of course there are other cases that could be aided by recognizing a notion of closed ranges. For example, `iota_view` can't in general represent a range that contains the maximum of its value type, because there's no valid past-the-end value for its iterator in that case.

But this notion of closed ranges is, as discussed above, foreign to the current C++ iterator model, and it is entirely unsurprising that `counted_iterator` isn't designed for it.

§ 5 Adapting a closed range into a half-open range

But let's say we have a closed range of N elements for whatever reason and want to adapt it to a half-open range that everything else in C++ expects. What do we need to do?

Well, we have N iterator values to start with and have to come up with $N + 1$ ones for the half-open range. In other words, we need to invent a new iterator value out of thin air. The adapted iterator has a single type that needs to represent both the original positions and this past-the-end value everything else expects - it is effectively a `variant<OriginalIterator, PastTheEndSentinel>` - though perhaps not literally that.

It should be unsurprising that such an adaptation necessarily adds overhead and limits functionality - every increment needs to check "did I become the sentinel"; every decrement - if supported at all - would need to check "am I the sentinel". And supporting decrement at all requires the `PastTheEndSentinel` to contain an `OriginalIterator` value. Returning non-`void` from postfix increment on input or output iterators cannot be supported - those can return a proxy, and we can't invent one out of thin air. And since output iterators require `*o++` to work, we lose support for those iterators entirely.

It's worth emphasizing that this problem has nothing to do with having a count. Adapting a closed range to a half-open range can be done without one. Presumably the reason this first arose in the `counted_iterator` context is that having a decrementing count alongside the iterator happens to provide a natural representation for the `PastTheEndSentinel` state. On the other hand, having a count also leads to a confusing iterator construction API if decrement is to be supported - especially if the adaptor can be used with normal C++ iterators where support for past-the-end values is the norm.

As a concrete example, consider range-v3's `closed_iota`, which represents an integer range `[first, last]`. Its iterator is basically

```
struct iterator {
    I current;
    I last;
```

```
bool past_the_end;  
};
```

where `operator++` sets `past_the_end` to `true` (and does not increment `current`) when `current == last`.

§ 6 Whatever the right approach for adapting a closed range into a half-open range is, `counted_iterator` is not it

Closed ranges are not a common occurrence in C++. How can they be, when types with no past-the-end values aren't even iterator types and every algorithm in the standard expects half-open ranges?

There are certainly interesting uses of closed ranges, and relaxing the requirement for a past-the-end value naturally means that you can express more things. It may well be worthwhile to add a closed-to-half-open adaptation mechanism so that such ranges can be used with other things in the standard library.

But `counted_iterator` should not be that mechanism. It does the job it is designed for well.⁴ Making a breaking change that adds performance overhead, inhibits compiler optimizations and limits functionality - at the last minute before we ship C++23 no less - is simply the wrong way to support closed ranges.

§ 7 Conclusion

There's nothing wrong with the design of `counted_iterator` for the use cases it was designed for.

If we want a facility to adapt a fully-closed range to a half-open range, we can add one in C++26. That's not `counted_iterator`'s job.

§ 8 References

[LWG2471] Jonathan Wakely. `copy_n`'s number of `InputIterator` increments unspecified.
<https://wg21.link/lwg2471>

[P2406R5] Yehezkel Bernat, Yehuda Bernat. 2023. Add `lazy_counted_iterator`.
<https://wg21.link/P2406R5>

-
1. “range” is in scary quotes because this combination does not actually meet the semantic requirements (due to the lack of a past-the-end value), despite meeting the syntactic requirements.[↵]
 2. `range-v3` adds a workaround for this issue by providing a `cached()` member in its `istream_view`, so that the user can use `tv.base().cached()`.[↵]

3. It's worth noting - as explained eight years ago in [[LWG2471](#)] - that anything that “fixes” `istream_iterator` (and `istream_view` - the two are indistinguishable for this purpose) - will likely break comparable uses of `istreambuf_iterator` instead.[↩](#)
4. It should be very clear from the discussion in this paper that the author strongly disagrees with any suggestion that `counted_iterator` “has the wrong default” or that the proposed `lazy_take`/`lazy_counted_iterator` should be used by default.[↩](#)